

Edge Signal-Processing Budget Attribution

What blocks real-time throughput, and the honest optimization path

2i

2026-07-29

Abstract

Before an edge core promises ‘real time,’ the promise needs to be measured against an actual nanosecond budget. This paper benchmarks a classical, 12-stage RF receiver front-end (synchronization, channel estimation, equalization, channelization, occupancy detection, interference cancellation, feature extraction, anomaly scoring, calibration, beamforming) on a single thread of a single core (Apple M3 Pro, aarch64, release build), and attributes throughput budget stage by stage. Aggregate throughput on a quiet machine averaged 7.59 MSPS, only 0.19x of a 40 MSPS target. The top three stages (RF feature extraction 35%, MVDR beamforming 29%, adaptive interference cancellation 21%) accounted for 85% of the budget, and applying optimistic, source-read-derived implementation optimizations to all three still capped aggregate throughput at 13.23 MSPS, or 0.33x of target. The conclusion is unambiguous: code optimization alone cannot reach 40 MSPS on this core. What is required is multiple cores (3 to 4 after optimization, 6 without), DSP/FPGA offload, or a faster single core. A separate neural-network pruning experiment reinforces the same accounting discipline from another angle: cutting a classifier’s channel width by 90% cut its FLOPs by 98.8% but its measured latency by only 38.5%. That FLOP reduction does not translate one-to-one into latency reduction is, again, something only measurement can tell you, not estimation.

Contents

1	Abstract	3
2	Why Measure the Edge Budget Honestly, Before Promising Real Time	3
3	Method: Per-Stage Throughput Attribution and Profiling	3
4	Measured Results	4
4.1	Aggregate Throughput and the Gap to Target	4
4.2	Per-Stage Budget Attribution	4
5	The Optimization Path, and Its Limits	6
5.1	The Top Three Stages: What Is Waste, What Is a Physical Floor	6
5.2	40 MSPS Is Still Out of Reach After Optimization	6
5.3	What Pruning Shows: FLOP Reduction Does Not Translate One-to-One Into Latency Reduction	7
6	Reproducibility	7
7	References	8

1 Abstract

Before an edge core promises “real time,” the promise needs to be measured against an actual nanosecond budget. This paper benchmarks a classical, 12-stage RF receiver front-end (synchronization, channel estimation, equalization, channelization, occupancy detection, interference cancellation, feature extraction, anomaly scoring, calibration, beamforming) on a single thread of a single core (Apple M3 Pro, aarch64, release build), and attributes throughput budget stage by stage. Aggregate throughput on a quiet machine averaged 7.59 MSPS, only 0.19x of a 40 MSPS target. The top three stages (RF feature extraction 35%, MVDR beamforming 29%, adaptive interference cancellation 21%) accounted for 85% of the budget, and applying optimistic, source-read-derived implementation optimizations to all three still capped aggregate throughput at 13.23 MSPS, or 0.33x of target. The conclusion is unambiguous: code optimization alone cannot reach 40 MSPS on this core. What is required is multiple cores (3 to 4 after optimization, 6 without), DSP/FPGA offload, or a faster single core. A separate neural-network pruning experiment reinforces the same accounting discipline from another angle: cutting a classifier’s channel width by 90% cut its FLOPs by 98.8% but its measured latency by only 38.5%. That FLOP reduction does not translate one-to-one into latency reduction is, again, something only measurement can tell you, not estimation.

2 Why Measure the Edge Budget Honestly, Before Promising Real Time

In the sales pitch for a wireless signal-processing product, the word “real time” is used cheaply. Promising to ingest a stream of tens of megasamples per second (MSPS) on a single cheap edge core is common; verifying that the promise actually sits on a real nanosecond budget is not. The gap matters for a simple reason. If the difference between target and measured throughput is 20 to 30 percent, profiling and tuning can close it. If the gap is more than five times, no amount of code polishing will reach a target that is physically out of reach on that hardware. The only way to tell these two situations apart is to measure the actual cost of each stage and determine whether that cost sits at an algorithmic floor or is implementation waste that can be removed.

The starting question for this work was concrete. Running a representative 12-stage pipeline - Schmidl-Cox timing synchronization, Moose carrier-frequency-offset estimation, least-squares (LS) channel estimation, zero-forcing (ZF) equalization, nearest-symbol detection, polyphase channelization, Urkowitz occupancy detection, NLMS adaptive-line-enhancer interference cancellation, RF fingerprinting feature extraction, Mahalanobis/cosine scoring, temperature/MSP calibration, and MVDR beamforming - on one cheap edge core, where exactly does the 5.3x budget gap to a 40 MSPS target live? Without answering that question, neither “optimize this one stage and you will hit the target” nor “code optimization cannot help, you need new hardware” is a claim anyone can actually back up.

This paper makes three contributions. First, it measures the full 12-stage pipeline repeatedly and ranks each stage’s share of the throughput budget. Second, for the top three stages, it reads the source directly to separate what is algorithmically required (a floor) from what implementation left on the table (waste). Third, it plugs the reproducible optimization headroom derived from that separation into a throughput projection to answer, directly and negatively, whether code optimization alone can reach 40 MSPS. A separate neural-network pruning experiment then shows that the same honest-accounting principle is not limited to the classical DSP chain.

3 Method: Per-Stage Throughput Attribution and Profiling

The measurement target is aarch64-apple-darwin (Apple M3 Pro, one of 12 logical cores used), release build, single thread. This is a development-laptop proxy for the target embedded ARM SoC - useful for relative budgeting and order-of-magnitude sizing, not a certified number for the shipping hardware.

Repetition and noise control. Each stage’s throughput was measured by rerunning the full pipeline build independently 15 times (each run redoing its own warmup and iterations from scratch), reporting both the mean and the coefficient of variation (CV). This workspace is a development machine with other processes sometimes competing for CPU, so load average and concurrent-process counts were measured via `sysctl` before and after each run and recorded honestly. The canonical run, taken in a window with no other heavy build or compute

jobs running, achieved an aggregate- throughput CV of 0.38%, with the top four stages all at or below 1.2% CV. By contrast, a preliminary run taken while another numerical workload was active on the same machine showed aggregate CV of 3.0 to 10.2% and per-stage CV as high as 36% - that data was kept only as a reference, not adopted as canonical.

Normalization unit. Because each stage’s single call processes a different unit of work (samples, windows, snapshots), every stage’s cost was normalized to nanoseconds per equivalent input sample (ns/sample) for fair comparison. Aggregate pipeline throughput is defined as the reciprocal of the sum of these 12 values ($1/\text{sum}(\text{ns_per_sample})$), the same quantity actually used to compute the throughput ceiling.

Robustness of the ranking. The set and order of the top three stages was identical across four independent runs at different repetition counts (15/15/15/25/20) and different load conditions. A separate session’s independent 8-repetition rerun, taken under roughly twice the load average of the canonical run (6.15 versus 3.36), still kept the top-three budget shares within 0.3 percentage points and measured aggregate throughput of 7.655 MSPS, within 1.1% of the canonical figure. Load visibly affects absolute throughput, as expected, but the *ratio* question - which stage is eating the budget - was far less sensitive to it. This paper’s conclusions rest on that relative attribution.

Confidence level of the headroom analysis. Optimization-headroom estimates for the top three stages are architecture-level estimates from reading the source directly, not real profiler (perf, Instruments) traces - elevated privileges required for those tools were not available in this environment. The estimates were cross-checked, however, against two measured quantities: heap allocation counts and heap high-water deltas (both measured with a dedicated global-allocator wrapper), which matched the number of new buffers counted directly in the source and their expected sizes. So the headroom multipliers themselves should be read as order-of-magnitude, but the allocation patterns behind them are measured, not estimated.

4 Measured Results

4.1 Aggregate Throughput and the Gap to Target

On a quiet machine, the canonical datasheet run measured aggregate throughput of **7.59 MSPS**. Against a 40 MSPS target, the realtime margin is **0.19x**, meaning at least 5.3x more single-core throughput is needed. A separately timed bottleneck- attribution canonical run, taken at a different time under different noise conditions, measured 7.460 MSPS (CV 0.38%), within 1.7% of the datasheet figure - a useful cross- validation given the two runs share nothing but the code. Under concurrent load, observed throughput dropped as low as 3.5 MSPS (a 0.09x margin); this is measurement, not noise, and is itself an honest finding that this core’s single-thread budget is genuinely sensitive to system load.

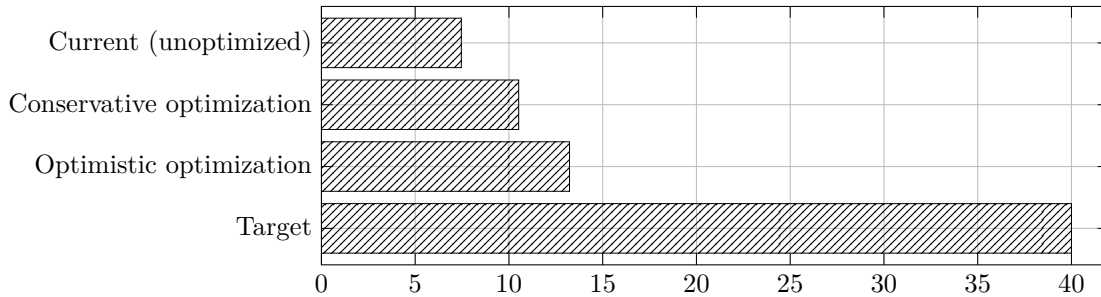


Figure 1: Aggregate throughput: current vs optimization scenarios vs target

Converting this gap into core count, and assuming the untested premise of linear scaling, reaching 40 MSPS with the code exactly as it stands today would require **about 6 cores**. Section 4 revisits why this linear-scaling assumption was not measured here and why not all 12 stages are equally parallelizable.

4.2 Per-Stage Budget Attribution

Ranking all 12 stages by normalized cost (canonical run, 15 repetitions, CV at or below 1.2%) gives:

Rank	Stage	ns/sample	Share	Cumulative
1	RF feature extraction (RFFI)	46.88	34.97%	34.97%
2	MVDR beamforming weight update + apply	39.52	29.48%	64.45%
3	NLMS adaptive interference cancellation (ALE)	27.93	20.83%	85.28%
4	Polyphase channelizer	7.59	5.66%	90.94%
5	Nearest-symbol detection (EVM/BER)	3.91	2.91%	93.86%
6	Schmidl-Cox timing synchronization	2.41	1.80%	95.65%
7	Mahalanobis/cosine scorer	1.79	1.33%	96.99%
8	LS channel estimation	1.30	0.97%	97.96%
9	ZF equalization	1.30	0.97%	98.92%
10	Urkowitz occupancy detection	0.74	0.55%	99.47%
11	Temperature/MSP calibration	0.68	0.51%	99.98%
12	Moose CFO estimation	0.02	0.02%	100.00%

The top three stages account for **85.28%** of the budget, and the drop-off past rank 4 (polyphase channelizer, 5.66%) is sharp - the (mean $\pm$ 2 standard deviation) bands of rank 3 and rank 4 do not overlap at all.

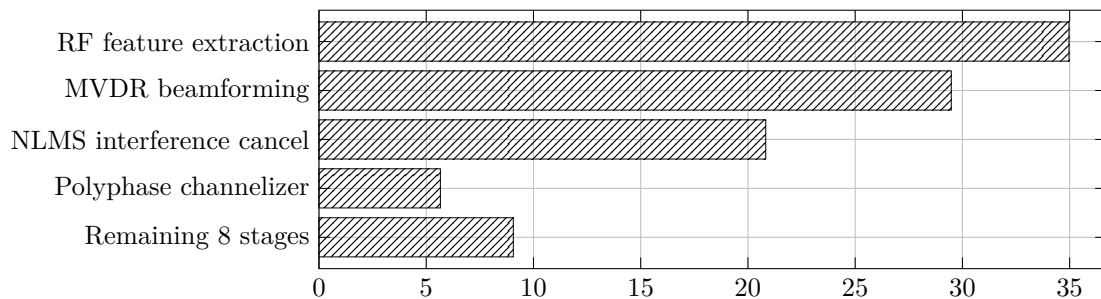


Figure 2: Per-stage budget share

This ranking was robust to noise. Across four independent runs with different repetition counts and load conditions (15/15/15/25/20 repetitions, including conditions where individual-stage CV ranged from 0.4% to 10.2%), the set and order of the top three stages was identical, and a rerun under roughly double the load kept budget shares within 0.3 percentage points. Absolute throughput moves with system load; the answer to “what is eating the budget” does not.

5 The Optimization Path, and Its Limits

5.1 The Top Three Stages: What Is Waste, What Is a Physical Floor

RF feature extraction (35%) - implementation waste dominates, headroom is large. This stage consists of one FFT plus more than 25 separate $O(W)$ linear scans (IQ imbalance, DC offset, frequency offset, envelope, peak-to-average-power ratio, phase derivative, six cumulants, five spectral features). A concrete, source-quotable waste exists here: the mean of a single envelope signal is independently recomputed inside the standard-deviation, skewness, and kurtosis calculations, so four statistics cost nine $O(W)$ passes. Other signal groups (instantaneous frequency, phase derivative) repeat the same pattern. Replacing this with a standard single-pass combined-moment technique (accumulating the sum, sum-of-squares, sum-of-cubes, and sum-of-fourth-powers in one pass and deriving mean/variance/skewness/kurtosis algebraically) can cut these groups' scan count to a third or less. The allocation count cross-validates this: the stage triggers an average of 11.7 heap allocations per call, closely matching the 10 new buffers counted directly in the source per call. The FFT itself is modest, $O(W \log W)$; most of the rest is this kind of redundant computation and allocation overhead. The reproducible headroom estimate is **roughly 2.0x to 3.0x**.

MVDR beamforming (29%) - algorithmic floor dominates, headroom is small. Covariance estimation and weight application are both $O(NM)$ in *snapshot count* (N) and *array element count* (M), and these two terms account for most of the stage's cost. There is an important negative result here: slowing the covariance update cadence does not reduce the per-snapshot cost, because the dominant terms scale with the number of snapshots processed, not with how often the result is used - only the negligible $O(M^3)$ matrix-inversion term benefits from a slower cadence. So "update less often" is not an effective lever for this stage. Implementation waste does exist alongside the floor: a small, effectively fixed-size (roughly 8×8) matrix is freshly heap-allocated on every call as a `Vec<Vec<complex>>`, producing about 22 heap allocations measured per call. Replacing this with a stack-allocated fixed array removes the allocations, but the underlying $O(NM)$ terms remain. The reproducible headroom estimate is **roughly 1.3x to 1.6x**.

NLMS adaptive interference cancellation (21%) - algorithmic floor dominates, headroom is small. Per-sample cost is $O(\text{order})$ complex multiply-accumulates, where order (16 taps) is a design parameter that sets how wide a band of narrowband interference the filter can track; shrinking it degrades suppression performance, so it is algorithmically necessary. Looking at measured heap usage, however, most of this stage's heap footprint is not filter state (tap weights) but two freshly allocated output buffers per call (4096 samples each, 128 KB total). A production deployment that reuses the canceller and writes into caller-owned buffers can remove this allocation entirely, and the 16-tap inner-product loop has unexplored SIMD headroom (not implemented or measured in this session). The reproducible headroom estimate is **roughly 1.3x to 1.8x**.

5.2 40 MSPS Is Still Out of Reach After Optimization

The current total budget is the sum of all 12 stages, 134.05 ns/sample ($= 1 / 7.460\text{e6}$ seconds), and reaching 40 MSPS requires that figure to drop to 25.00 ns/sample or below - a 5.36x improvement from today. Applying the headroom above only to the top three stages (the bottom nine already account for just 14.72% of the budget) gives a conservative and an optimistic bound:

Scenario	Total budget (ns/sample)	Aggregate throughput	vs. 40 MSPS target	Cores required
Current (unoptimized)	134.07	7.46 MSPS	0.186x	6
Conservative optimization	95.06	10.52 MSPS	0.263x	4
Optimistic optimization	75.59	13.23 MSPS	0.331x	3

The answer is no. Even the optimistic scenario, where all three top stages hit their best-case optimization simultaneously, reaches only 0.33x of the target - still short by more than 3x. "40 MSPS through code optimization alone" does not hold on this core. What is needed is multiple cores (3 to 4 after optimization, 6 without

any optimization), a faster single core, or DSP/FPGA offload. To be candid, the linear- core-scaling assumption above was not measured in this work, and not all 12 stages parallelize equally well: RF feature extraction, MVDR beamforming, and polyphase channelization are independent per window and thus easy to data-parallelize, but NLMS interference cancellation carries adaptive filter state across blocks in a streaming fashion, so naively splitting the time axis across cores would break filter continuity - distributing whole pipeline stages across cores is a safer structure than naive data parallelism. Notably, the top three stages are also textbook DSP-offload candidates - an FFT (RF feature extraction), matrix arithmetic (MVDR), and an FIR/adaptive filter (NLMS) are exactly the operation types small DSP coprocessors or FPGA fabric are typically specialized for.

5.3 What Pruning Shows: FLOP Reduction Does Not Translate One-to-One Into Latency Reduction

To check whether the same honest-accounting principle applies to a neural-network component as well, a separate, compact modulation-classification network (a 1D CNN with hidden widths 64/128/128/256) was structurally channel-pruned, retrained, and timed on CPU. This is a different component from the 12-stage classical DSP chain in Section 3, but it answers a question that comes up naturally whenever a signal- processing front-end pairs with a learned classifier: does cutting FLOPs actually make it faster?

Configuration	Accuracy (post-finetune)	Parameters	MACs	Latency (mean, CPU)
Dense (widths 64/128/128/256)	93.91%	193,867	5,884,672	0.1181 ms
50% width reduction	91.64% (-2.27 pp)	49,835 (-74.3%)	1,500,544 (-74.5%)	0.1027 ms (-13.1%)
90% width reduction	81.27% (-12.64 pp)	2,466 (-98.7%)	68,446 (-98.8%)	0.0726 ms (-38.5%)

Halving the width cuts FLOPs by 74.5% but latency by only 13.1%; cutting width by 90% cuts FLOPs by 98.8% but latency by only 38.5%. In both cases, pre-finetune accuracy briefly collapsed to a chance-level 9.09% and was recovered by retraining, and the 90% reduction trades 12.6 percentage points of accuracy for that latency gain. The large gap between the FLOP-reduction percentage and the latency-reduction percentage means that, at this small model scale, CPU inference time is driven more by memory access and call overhead than by raw arithmetic - reading FLOP counts alone would overstate the optimization’s real-world payoff. This is the same lesson repeated from Section 3 and 4.1: the budget is the wall-clock time actually measured, not a theoretical operation count or a stated reduction percentage.

6 Reproducibility

This paper’s per-stage attribution table is also generated as machine-readable JSON, with rank, sums, and coefficients of variation all computed by code (the prose only describes the result). The reproduction commands are:

```
# Bottleneck attribution (15 repetitions, release build)
EDGE BENCH_BOTTLENECK_REPEATS=15 cargo test --release -p edgebench --test bottlenecks -- --nocapture

# Structural invariant checks (contiguous rank 1..N, shares sum to 100%, top-3 share range)
cargo test --release -p edgebench --lib -- --nocapture

# 300-second sustained-run datasheet (throughput / latency / memory / leak heuristic)
EDGE BENCH_SUSTAINED_MIN_S=300 cargo test --release -p edgebench -- --nocapture
```

No new external dependency was added for measurement. Heap instrumentation uses a dedicated `#[global_allocator]` wrapper; process peak resident-set size (RSS) was obtained via a direct `getrusage(RUSAGE_SELF)` FFI call. The pruning experiment (Section 4.3) comes from a separate scaling-bench study, using the same

data split (9,900 training / 1,100 test examples), retrained at each width-reduction ratio (50%, 90%) with accuracy and CPU latency measured together.

This work is also honest about what it did not measure: on-target embedded ARM SoC performance (only a development-laptop proxy was run), fine-grained instruction-level attribution from a real profiler (source-read estimates were substituted, since perf and Instruments both require elevated privileges unavailable in this environment), measured multicore scaling (only the linear-scaling assumption was used), and implementation plus post-change remeasurement of the code changes proposed in Section 4.1. This paper’s deliverable is an honest map of attribution and its limits; the code changes themselves are left as a next step that each component owner can pick up independently on top of that map.

7 References

1. E. J. Kelly, “An Adaptive Detection Algorithm,” *IEEE Transactions on Aerospace and Electronic Systems*, vol. AES-22, no. 2, pp. 115-127, 1986. (Background for MVDR/adaptive beamforming.)
2. B. Widrow and S. D. Stearns, *Adaptive Signal Processing*, Prentice-Hall, 1985. (Background for NLMS adaptive line enhancement.)
3. M. Schmidl and D. Cox, “Robust Frequency and Timing Synchronization for OFDM,” *IEEE Transactions on Communications*, vol. 45, no. 12, pp. 1613-1621, 1997.
4. H. Urkowitz, “Energy Detection of Unknown Deterministic Signals,” *Proceedings of the IEEE*, vol. 55, no. 4, pp. 523-531, 1967. (Background for occupancy/energy detection.)
5. S. Han, H. Mao, and W. J. Dally, “Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding,” *ICLR*, 2016. (Background for structured pruning.)
6. The Rust Project, “rustfft,” and the project’s own instrumentation code (`#[global_allocator]` wrapper, `getrusage FFI`); see Section 5 for reproduction commands.